

Development · UAT · Production Plan

Merge strategy, secure code practices, versioning, and rollback

Scope: **Frontend** + **API** + **Database**

Project: CCSHAU_Project / ccshau-website

Repo: github.com/SiyanainfoAhm/ccshau-website

Stack: Next.js (apps/web) · Supabase Postgres · Vercel

Document: v1.0 · 8 September 2026

Environments: Development · UAT · Production

Table of Contents

1. Current state (review findings)
2. Target environment model
3. Architecture diagrams
4. Git merge & versioning plan
5. Frontend plan (Next.js / Vercel)
6. API plan (Server Actions + Route Handlers)
7. Database plan (Supabase migrations)
8. Secure code practices
9. Promotion pipeline (step-by-step)
10. Rollback plan (Frontend / API / Database)
11. Release checklist
12. Phased implementation roadmap

1. Current state (review findings)

Area	What exists today	Gap
Frontend	Next.js 16 monorepo at <code>apps/web</code> ; Vercel Production on <code>main</code> ; PR previews	No dedicated UAT branch/domain mapping
API	Server Actions + thin routes (<code>/api/auth</code> , <code>cron</code> , <code>downloads</code> , <code>health</code>)	Same codebase deploys with frontend; no env-isolated API secrets matrix documented for 3 envs
Database	<code>supabase/migrations/</code> (~60+ files); manual <code>supabase db push</code> ; Pro backups	Single shared cloud project historically; Dev/UAT not fully isolated
CI	GitHub Actions lint/test/build on <code>main</code> and <code>develop</code>	<code>develop</code> / <code>uat</code> branches not fully established as promotion path
Rollback	Vercel previous deploy + DB daily backups	Need explicit version-linked rollback SOP for FE + API + DB together

Goal: Every Production release is a tagged version (`vx.y.z`) that includes the same Frontend + API code commit and matching Database migration set — with a documented rollback path if anything fails.

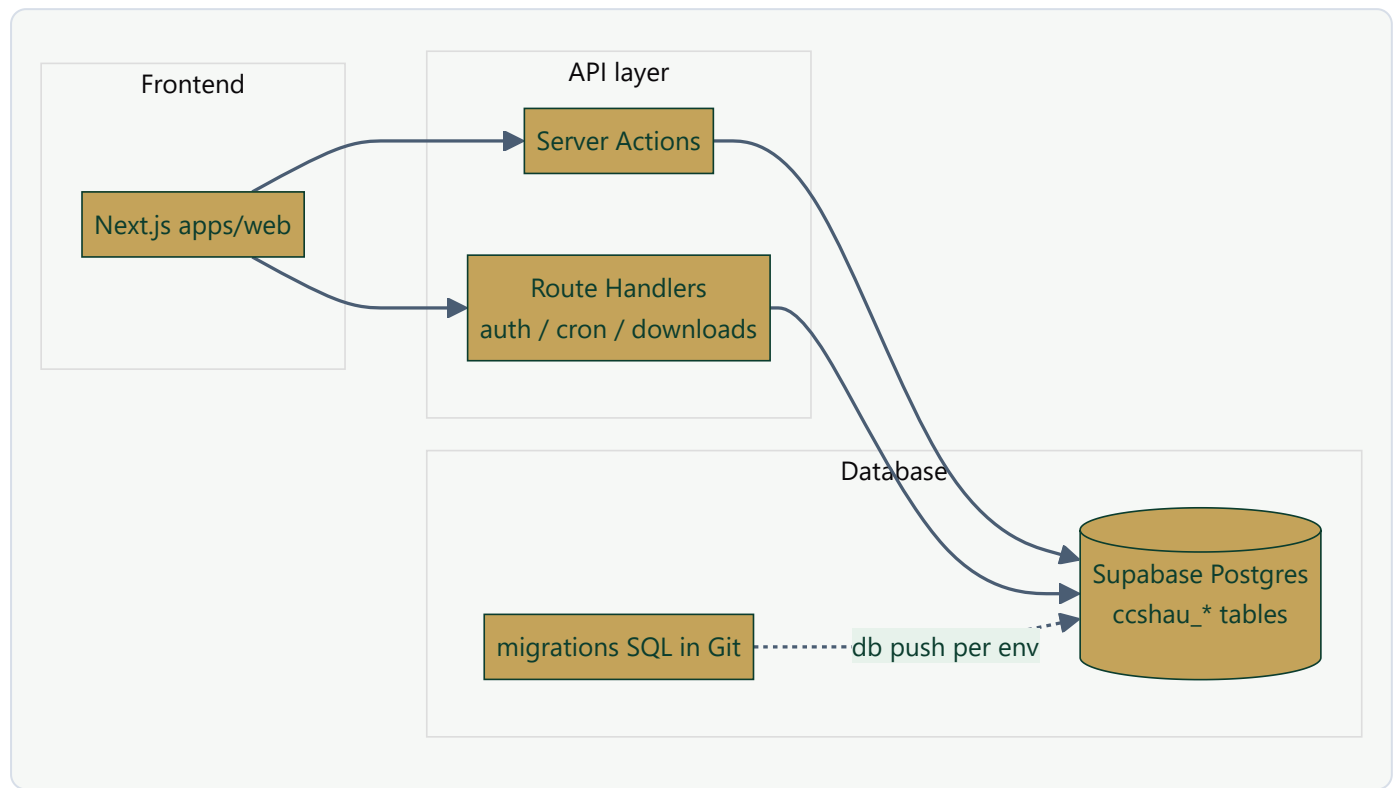
2. Target environment model

Environment	Git branch	Frontend host	API (same Next app)	Database
Development	develop / feature/*	Local + Vercel Preview/Dev URL	Server Actions / routes on Dev deploy	Supabase Dev project
UAT	uat	Vercel UAT domain (e.g. uat....)	Same app on UAT deploy	Supabase UAT project
Production	main + tag vX.Y.Z	Vercel Production (live domain)	Same app on Production	Supabase Production project

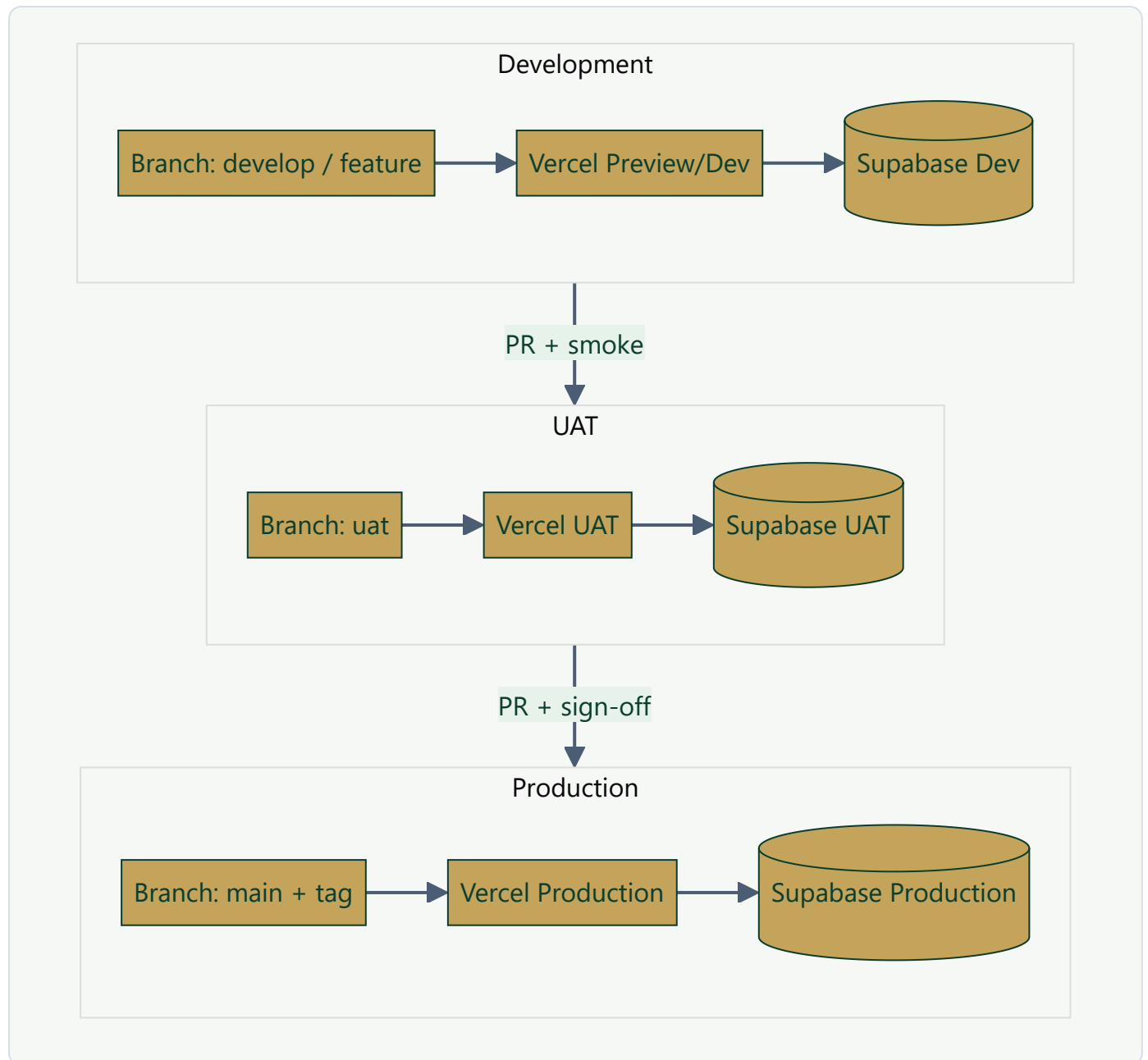
Isolation rule: Do not point Dev/UAT frontends at the Production Supabase project. Separate projects (or at minimum separate schemas + keys) prevent accidental CMS/data damage.

3. Architecture diagrams

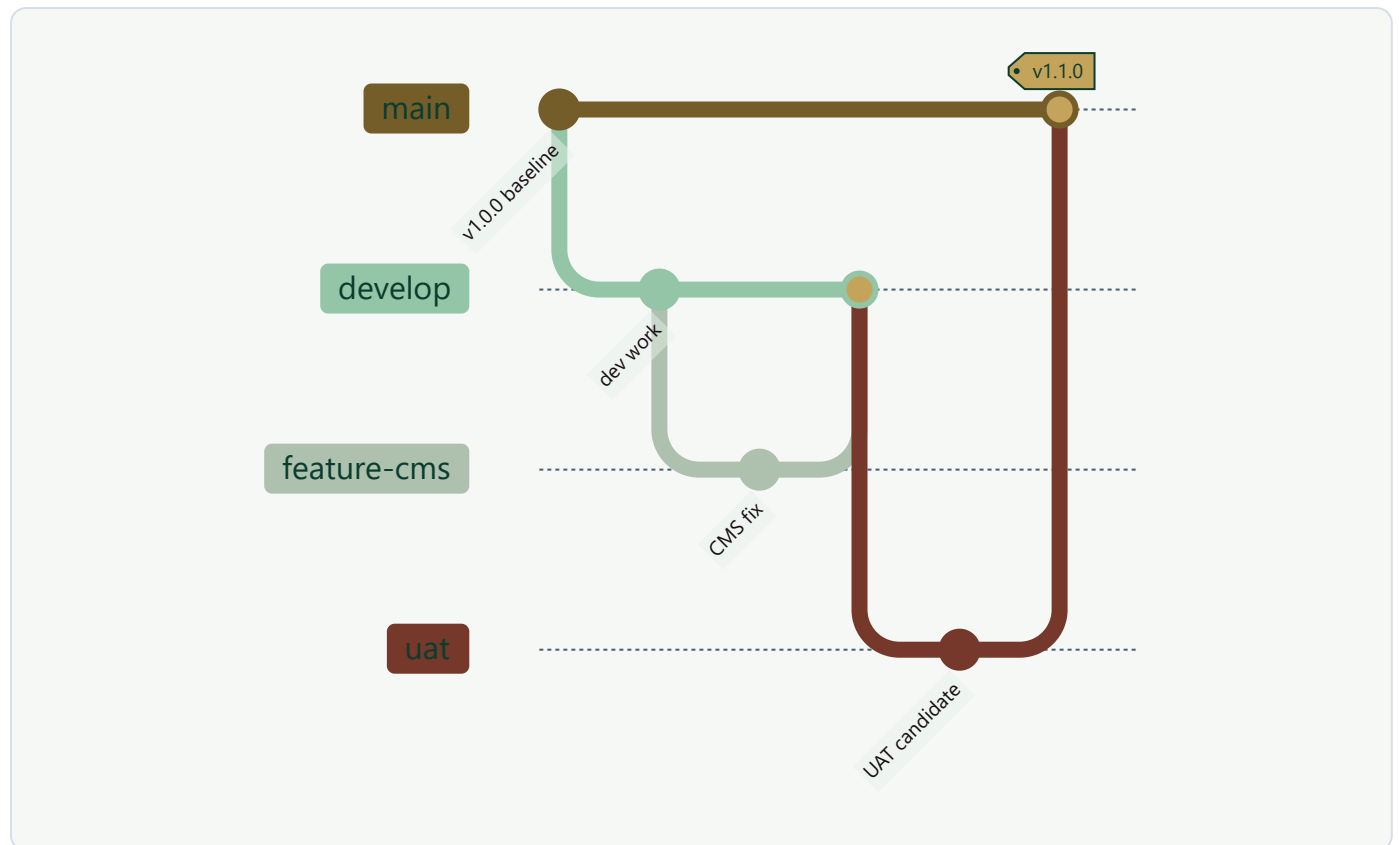
3.1 Frontend + API + Database per environment



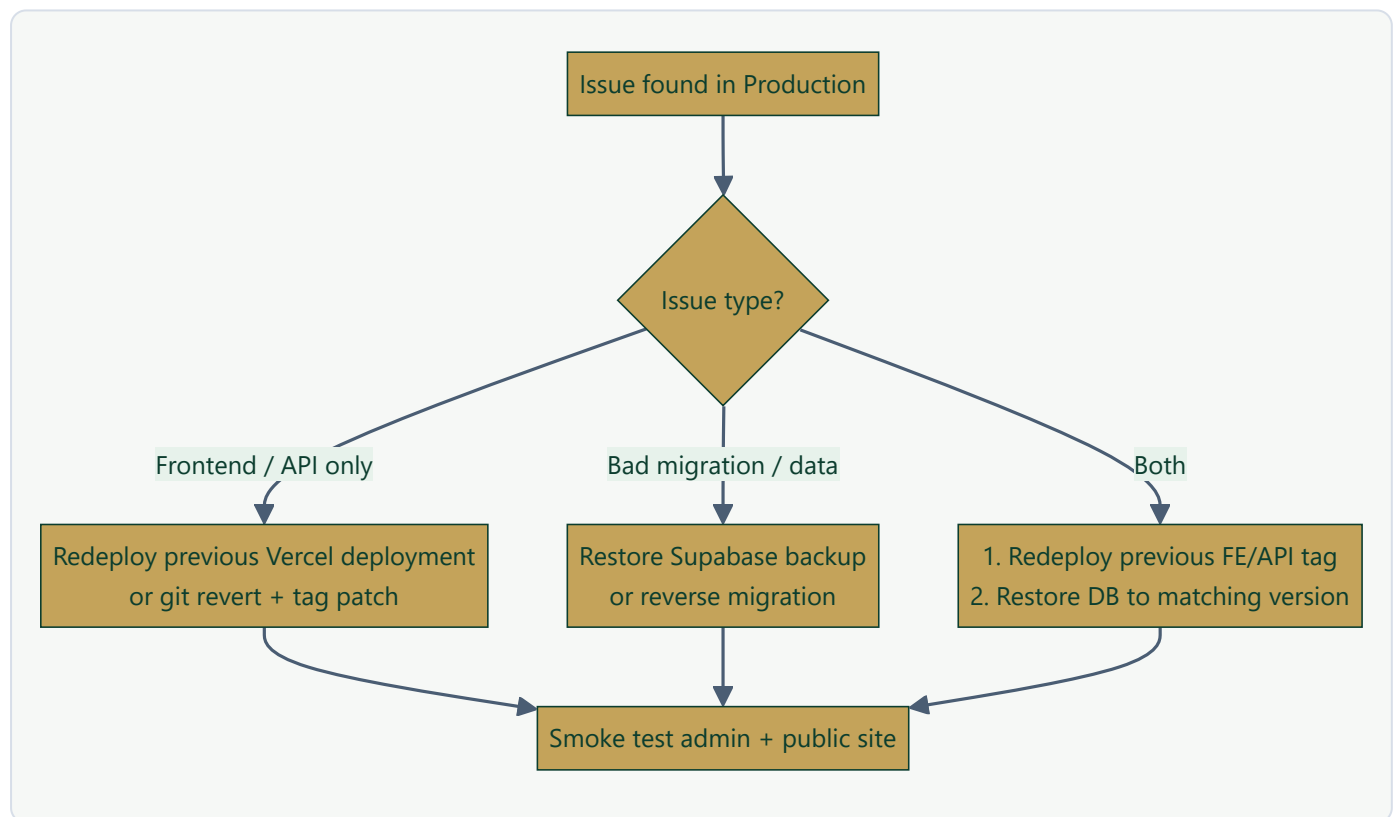
3.2 Three environments (isolated)



3.3 Merge flow

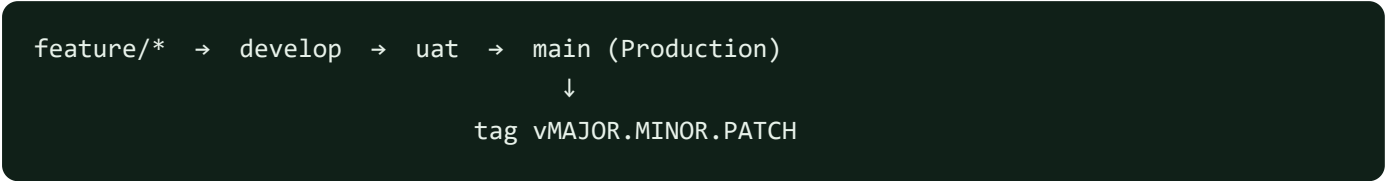


3.4 Rollback overview



4. Git merge & versioning plan

4.1 Branch strategy



Branch	Purpose	Who merges	Deploy
feature/*	Single change / ticket	Developer via PR → develop	Optional preview
develop	Integration / Development	Team	Dev environment
uat	Frozen UAT candidate	Lead via PR from develop	UAT environment
main	Production only	Lead via PR from uat	Production + tag

4.2 Merge rules (secure process)

- No direct push to `uat` or `main` — Pull Requests only
- Require CI green (lint + test + build) before merge
- Require at least 1 code review on PRs into `uat` / `main`
- Block force-push on `uat` and `main`
- Keep repository **private**; least-privilege collaborator roles
- Never commit secrets; enable GitHub secret scanning / push protection

4.3 Semantic versioning

Version part	When to bump	Example
MAJOR	Breaking schema/API/frontend contract	v2.0.0
MINOR	New feature (CMS module, page type)	v1.4.0
PATCH	Bugfix / security patch / hotfix	v1.4.1

Version package: Tag `vX.Y.Z` = Frontend commit + API (same commit) + list of migration filenames applied for that release (recorded in release notes).

4.4 Hotfix merge path

```
main → hotfix/xxx → PR into main → tag vX.Y.Z+1 (patch)  
    ↘ merge hotfix back into uat and develop
```

5. Frontend plan (Next.js / Vercel)

1 Root directory: always deploy from monorepo with Vercel Root = `apps/web` .

2 Branch mapping: `develop` → Dev, `uat` → UAT, `main` → Production.

3 Env vars per environment (Vercel Development / Preview / Production):

Variable	Frontend?	Notes
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Yes	Different per env
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Yes	Different per env
<code>NEXT_PUBLIC_SITE_URL</code>	Yes	Env domain
<code>NEXT_PUBLIC_AZURE_STORAGE_*</code>	Yes	Prefer separate containers per env if possible
<code>NEXT_PUBLIC_DEV_PREFILL_LOGIN</code>	Yes	false on UAT/Production

4 Rollback (Frontend): Vercel → Deployments → previous successful Production deploy → *Promote / Redeploy*. Or revert git commit on `main` and push a patch tag.

6. API plan (Server Actions + Route Handlers)

In this project, API ships **with** the Next.js app (no separate Edge Functions folder).

API surface	Path	Secure practice
Server Actions	apps/web/src/actions/	Validate auth/RBAC server-side; never trust client role claims alone
Auth routes	src/app/api/auth/*	HttpOnly cookies / session via Supabase SSR
Cron	src/app/api/cron/*	Protect with CRON_SECRET ; Production only schedules
Downloads	src/app/api/downloads/[id]/file	Authorize before streaming
Health	src/app/api/health	No secrets in response

API secrets (server-only, per env)

- SUPABASE_SERVICE_ROLE_KEY — never expose to browser
- AZURE_STORAGE_CONNECTION_STRING
- CRON_SECRET
- POWER_AUTOMATE_* , CAPTCHA_SECRET_KEY (when enabled)

R Rollback (API): same as Frontend (redploy previous Vercel deployment / previous git tag).
API and Frontend share one deployment unit.

7. Database plan (Supabase)

7.1 Source of truth

```
supabase/migrations/*.sql ← versioned in Git  
Apply order: Dev → UAT → Production (same files)
```

7.2 Rules

- New change = **new** migration file (never edit applied migrations)
- Naming: timestamp + description (existing convention)
- Record which migrations are part of release `vX.Y.Z` in release notes
- Prefer additive/backward-compatible migrations when possible (easier rollback)

7.3 Apply steps (per environment)

```
# Link the target project, then push migrations  
npx supabase link --project-ref <ENV_PROJECT_REF>  
npx supabase db push
```

7.4 Data safety

- Dev/UAT use test/CMS sandbox content — not Production citizen/university live content copies unless anonymized
- Production: keep Supabase Pro daily backups enabled (already documented)
- Before risky Production migration: take a manual backup / note restore point time

8. Secure code practices

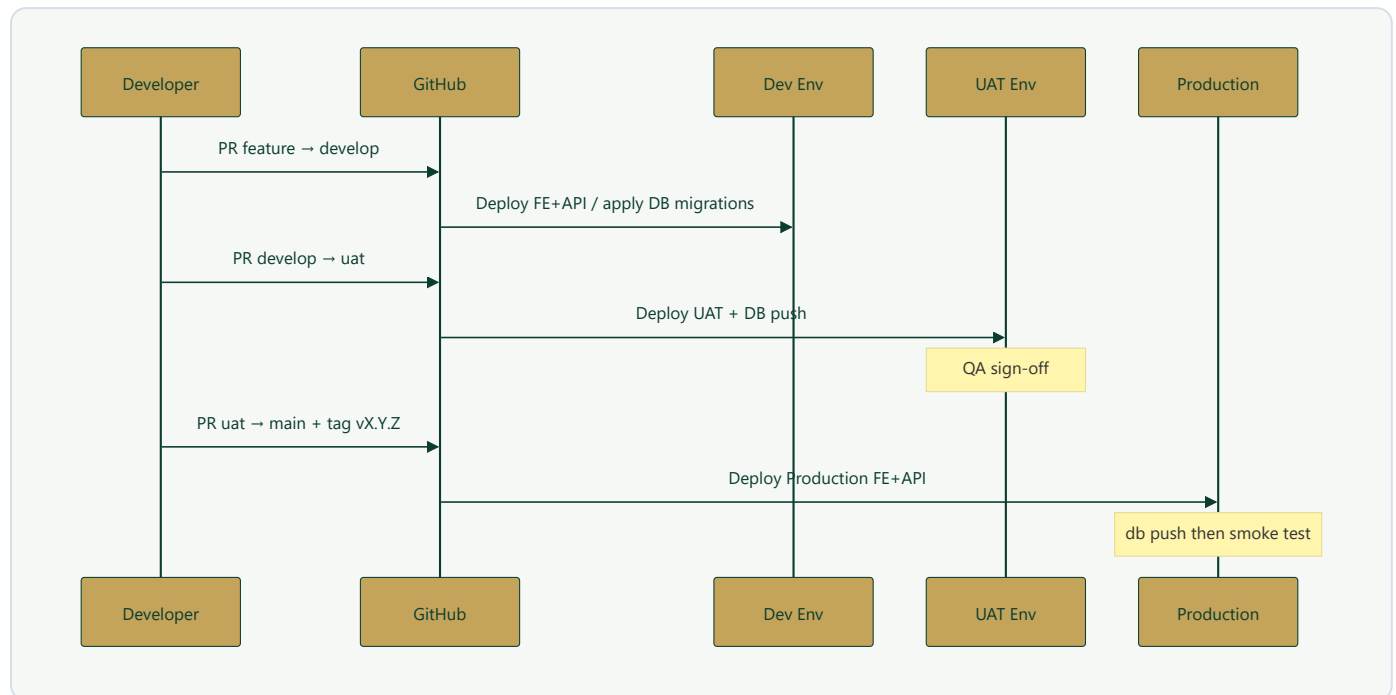
Repository & process

- Private GitHub repo
- Branch protection on `uat` / `main`
- PR + review + CI required
- Secret scanning / push protection
- Least-privilege collaborator access
- No force-push to protected branches

Code & secrets

- Never commit `.env.local` / real keys
- Only `.env.example` placeholders in Git
- Service role key server-only
- RLS on Supabase tables
- Validate inputs in Server Actions
- Disable Dev login prefill on UAT/Prod

9. Promotion pipeline (step-by-step)



1. Create `feature/...` from `develop` ; implement Frontend/API; add migration if needed.
2. PR → `develop` ; CI must pass; merge; deploy Dev; `db push` to Dev Supabase.
3. Smoke test Dev (public pages + admin actions).
4. PR `develop` → `uat` ; merge; deploy UAT; `db push` to UAT.
5. QA executes checklist; business/university UAT sign-off.
6. PR `uat` → `main` ; merge; create tag `vX.Y.Z` .
7. Deploy Vercel Production (auto from `main`).
8. Apply Production migrations (`db push`) **after** confirming release notes / backup point.
9. Production smoke test; mark release complete.

Order tip: For additive DB changes, deploy FE/API first then migrate. For breaking schema changes, migrate in a compatible two-step release (expand → deploy code → contract), or take a maintenance window with clear rollback.

10. Rollback plan (if issue found)

10.1 Decide what failed

Symptom	Likely layer	First action
UI/admin broken, DB data OK	Frontend / API	Redeploy previous Vercel Production deployment
API errors / 500s after deploy	API / env vars	Check logs + redeploy previous tag
Data corruption / failed migration	Database	Restore backup or run reverse migration
Everything broken after release	All	Rollback FE/API to previous tag and restore DB to pre-release backup

10.2 Frontend + API rollback (fastest)

1. Open Vercel → Project → Deployments.
2. Find last known-good Production deployment (or tag `vX.Y.(Z-1)`).
3. Promote/Redeploy that deployment to Production.
4. Verify `/api/health` , homepage, admin login, one CMS write.

Git alternative:

```
git checkout main
git revert <bad-commit> # or reset only if not shared / with approval
git push
# tag patch: vX.Y.Z+1-rollback
```

10.3 Database rollback

Preferred: reverse migration

If change is reversible, add a new migration that undoes it (drop column added, restore constraint, etc.). Apply via `db push` . Keeps history clean.

Emergency: restore backup

Supabase Dashboard → Database → Backups → restore to point before release. Follow `docs/ops/backup-restore-guide.md` . Expect downtime; coordinate.

Critical: After a DB restore, Frontend/API must run a version compatible with that schema. Always roll FE/API back to the tag that matches the restored DB version.

10.4 Version matching matrix (keep this in release notes)

Release tag	Git commit	Migrations included	Vercel deploy	Rollback target
v1.2.0	abc1234	2026....sql ...	Prod URL / deploy id	v1.1.3
v1.2.1	...	...	...	v1.2.0

11. Release checklist

Before UAT

- ☐ PR reviewed; CI green
- ☐ Migrations listed
- ☐ UAT env vars verified
- ☐ Dev smoke test passed

Before Production

- ☐ UAT signed off
- ☐ Production backup / restore point noted
- ☐ Release notes include tag + migrations + rollback target
- ☐ `NEXT_PUBLIC_DEV_PREFILL_LOGIN=false` on Prod
- ☐ Cron secrets present if cron used

After Production

- ☐ Health endpoint OK
- ☐ Public homepage + one college page OK
- ☐ Admin login + one content update OK
- ☐ Upload / Azure URL OK (if touched)
- ☐ Tag `vx.y.z` pushed to GitHub

If rollback needed

- ☐ Identify layer (FE/API vs DB vs both)
- ☐ Redeploy previous Vercel deployment / tag
- ☐ Reverse migration or restore DB if required
- ☐ Re-test checklist
- ☐ Post-incident note: cause + fix version

12. Phased implementation roadmap

Phase	Actions	Outcome
Phase 1	Create <code>develop</code> + <code>uat</code> from <code>main</code> ; protect <code>main</code> / <code>uat</code> ; document release notes template	Merge path exists
Phase 2	Create Dev + UAT Supabase projects; copy schema via migrations; wire Vercel env vars per branch	Isolated FE/API/DB
Phase 3	Enforce PR-only promotion; map Vercel Dev/UAT/Prod; start tagging Production releases	Versioned releases
Phase 4	Practice one rollback drill on UAT (redeploy + reverse migration)	Proven rollback

One-line summary: Promote Frontend + API + Database together through `develop` → `uat` → `main` , tag every Production release, keep secrets out of Git, and always retain a previous deploy + DB restore point for rollback.

CCSHAU Website — Environment, Merge, Security & Rollback Plan v1.0

Repo: <https://github.com/SiyanainfoAhm/ccshau-website>

Related: [Documents/ci-cd-step-by-step-guide.md](#) , [docs/ops/backup-restore-guide.md](#) , [supabase/migrations/](#) , [apps/web](#)